In [1]:
```python
import tensorflow as tf
import kagglehub
import os
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import ResNet50, VGG16, MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
```

In [2]:
```python
path = kagglehub.dataset_download("marquis03/flower-classification")

print("Dataset Path:", path)
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed:
0.3.13), please consider upgrading to the latest version (1.0.0).
Downloading from https://www.kaggle.com/api/v1/datasets/download/marquis03/flow
er-classification?dataset_version_number=1...
100%|██████████| 205M/205M [00:01<00:00, 173MB/s]
Extracting files...
Dataset Path: /root/.cache/kagglehub/datasets/marquis03/flower-classification/v
ersions/1
```

In [3]:
```python
dataset_path = path

print(os.listdir(dataset_path))
```

```
['val', 'val.csv', 'classname.txt', 'train', 'train.csv']
```

In [16]:
```python
train_path = os.path.join(dataset_path, "train")
val_path = os.path.join(dataset_path, "val")

print("Train folder:", train_path)
print("Validation folder:", val_path)
```

```
Train folder: /root/.cache/kagglehub/datasets/marquis03/flower-classification/v
ersions/1/train
Validation folder: /root/.cache/kagglehub/datasets/marquis03/flower-classificat
ion/versions/1/val
```

In [20]:
```python
class_names = [d for d in os.listdir(train_path)
               if os.path.isdir(os.path.join(train_path, d))]

print("Flower Classes:")
for i, name in enumerate(class_names):
    print(i, ":", name)

print("Total Classes:", len(class_names))
```

```
Flower Classes:
0 : water_lily
1 : iris
2 : dandelion
3 : california_poppy
4 : tulip
5 : astilbe
6 : bellflower
7 : sunflower
8 : coreopsis
9 : black_eyed_susan
10 : calendula
11 : rose
12 : common_daisy
13 : carnation
Total Classes: 14
```

In [22]:

```python
import random
import matplotlib.pyplot as plt
import math

num_classes = len(class_names)

cols = 3
rows = math.ceil(num_classes / cols)

plt.figure(figsize=(12, 4*rows))

for i, class_name in enumerate(class_names):

    class_path = os.path.join(train_path, class_name)

    img_name = random.choice(os.listdir(class_path))

    img_path = os.path.join(class_path, img_name)

    img = plt.imread(img_path)

    plt.subplot(rows, cols, i+1)
    plt.imshow(img)
    plt.title(class_name)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

water_lily

iris

dandelion

california_poppy

tulip

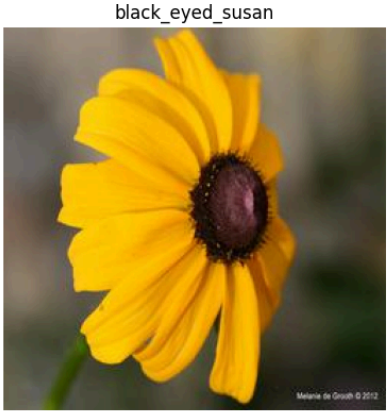
astilbe

bellflower

sunflower

coreopsis

black_eyed_susan



calendula

rose

common_daisy

carnation

In [23]:
```
print("Total Classes:", len(class_names))

for c in class_names:
    path = os.path.join(train_path, c)
    print(c, ":", len(os.listdir(path)), "images")
```

```
Total Classes: 14
water_lily : 977 images
iris : 1041 images
dandelion : 1038 images
california_poppy : 1021 images
tulip : 1034 images
astilbe : 726 images
bellflower : 872 images
sunflower : 1013 images
coreopsis : 1035 images
black_eyed_susan : 986 images
calendula : 1011 images
rose : 986 images
common_daisy : 978 images
carnation : 924 images
```

In [5]:
```
def build_model(base_model):

    base_model.trainable = False

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(128, activation='relu')(x)
    output = Dense(num_classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=output)

    model.compile(
        optimizer=Adam(learning_rate=0.0001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

In [6]:
```
resnet_base = ResNet50(weights='imagenet', include_top=False, input_shape=(224

resnet_model = build_model(resnet_base)

history_resnet = resnet_model.fit(
    train_data,
    validation_data=val_data,
    epochs=5
)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 58s 133ms/step - accuracy: 0.9782 - loss: 0.0744 -
val_accuracy: 0.9931 - val_loss: 0.0425
Epoch 2/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 38s 110ms/step - accuracy: 0.9928 - loss: 0.0430 -
val_accuracy: 0.9931 - val_loss: 0.0422
Epoch 3/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 36s 103ms/step - accuracy: 0.9928 - loss: 0.0430 -
val_accuracy: 0.9931 - val_loss: 0.0420
Epoch 4/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 37s 107ms/step - accuracy: 0.9928 - loss: 0.0430 -
val_accuracy: 0.9931 - val_loss: 0.0417
Epoch 5/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 39s 112ms/step - accuracy: 0.9928 - loss: 0.0429 -
val_accuracy: 0.9931 - val_loss: 0.0416
```

In [7]:

```
vgg_base = VGG16(weights='imagenet', include_top=False, input_shape=(224,224,3

vgg_model = build_model(vgg_base)

history_vgg = vgg_model.fit(
    train_data,
    validation_data=val_data,
    epochs=5
)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 123s 313ms/step - accuracy: 0.9713 - loss: 0.0946
- val_accuracy: 0.9931 - val_loss: 0.0438
Epoch 2/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 246ms/step - accuracy: 0.9928 - loss: 0.0447 -
val_accuracy: 0.9931 - val_loss: 0.0432
Epoch 3/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 248ms/step - accuracy: 0.9928 - loss: 0.0441 -
val_accuracy: 0.9931 - val_loss: 0.0430
Epoch 4/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 247ms/step - accuracy: 0.9928 - loss: 0.0436 -
val_accuracy: 0.9931 - val_loss: 0.0424
Epoch 5/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 84s 246ms/step - accuracy: 0.9928 - loss: 0.0433 -
val_accuracy: 0.9931 - val_loss: 0.0421
```

In [8]:

```
mobile_base = MobileNetV2(weights='imagenet', include_top=False, input_shape=(

mobile_model = build_model(mobile_base)

history_mobile = mobile_model.fit(
    train_data,
```

```
    validation_data=val_data,
    epochs=5
)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_to
p.h5
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 64s 138ms/step - accuracy: 0.9877 - loss: 0.0578 -
val_accuracy: 0.9931 - val_loss: 0.0442
Epoch 2/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 21s 62ms/step - accuracy: 0.9928 - loss: 0.0438 -
val_accuracy: 0.9931 - val_loss: 0.0432
Epoch 3/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 21s 61ms/step - accuracy: 0.9928 - loss: 0.0399 -
val_accuracy: 0.9931 - val_loss: 0.0435
Epoch 4/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 40s 59ms/step - accuracy: 0.9928 - loss: 0.0359 -
val_accuracy: 0.9931 - val_loss: 0.0443
Epoch 5/5
344/344 ━━━━━━━━━━━━━━━━━━━━ 20s 57ms/step - accuracy: 0.9928 - loss: 0.0323 -
val_accuracy: 0.9931 - val_loss: 0.0435
```

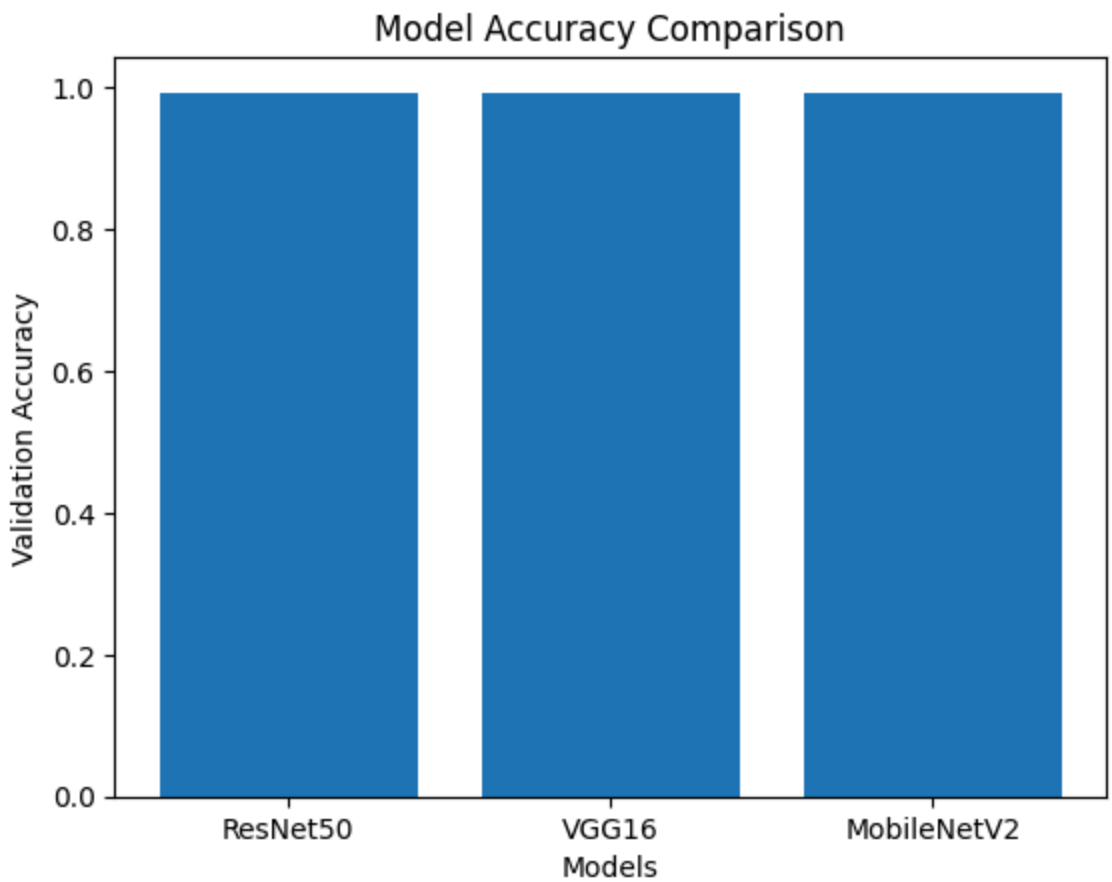
In [12]:

```
resnet_acc = history_resnet.history['val_accuracy'][-1]
vgg_acc = history_vgg.history['val_accuracy'][-1]
mobile_acc = history_mobile.history['val_accuracy'][-1]

print("ResNet50 Accuracy:", resnet_acc)
print("VGG16 Accuracy:", vgg_acc)
print("MobileNetV2 Accuracy:", mobile_acc)
```

```
ResNet50 Accuracy: 0.9930833578109741
VGG16 Accuracy: 0.9930833578109741
MobileNetV2 Accuracy: 0.9930833578109741
```

In [13]:

```
models = ['ResNet50','VGG16','MobileNetV2']
accuracy = [resnet_acc, vgg_acc, mobile_acc]

plt.bar(models, accuracy)
plt.title("Model Accuracy Comparison")
plt.ylabel("Validation Accuracy")
plt.xlabel("Models")
plt.show()
```



In [27]:
```
learning_rates = [0.0001, 0.00001]
dropouts = [0.3, 0.5]

epochs = 3
```

In [28]:
```
from tensorflow.keras.layers import Dropout

def build_model_hp(base_model, lr, dr):

    base_model.trainable = False

    x = base_model.output
    x = GlobalAveragePooling2D()(x)

    x = Dense(256, activation='relu')(x)
    x = Dropout(dr)(x)

    output = Dense(train_data.num_classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=output)

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='categorical_crossentropy',
        metrics=['accuracy']
```

```
    )

    return model
```

In [29]:

```
results = []

models = {
    "ResNet50": ResNet50,
    "VGG16": VGG16,
    "MobileNetV2": MobileNetV2
}

for model_name, model_class in models.items():

    for lr in learning_rates:
        for dr in dropouts:

            print("\nTraining:", model_name,
                  "| LR:", lr,
                  "| Dropout:", dr)

            base_model = model_class(
                weights='imagenet',
                include_top=False,
                input_shape=(224,224,3)
            )

            model = build_model_hp(base_model, lr, dr)

            history = model.fit(
                train_data,
                validation_data=val_data,
                epochs=3
            )

            acc = history.history['val_accuracy'][-1]

            results.append((model_name, lr, dr, acc))
```

```
Training: ResNet50 | LR: 0.0001 | Dropout: 0.3
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 54s 129ms/step - accuracy: 0.9804 - loss: 0.0680 -
val_accuracy: 0.9931 - val_loss: 0.0435
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 36s 105ms/step - accuracy: 0.9928 - loss: 0.0444 -
val_accuracy: 0.9931 - val_loss: 0.0417
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 38s 109ms/step - accuracy: 0.9928 - loss: 0.0454 -
val_accuracy: 0.9931 - val_loss: 0.0419

Training: ResNet50 | LR: 0.0001 | Dropout: 0.5
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 59s 141ms/step - accuracy: 0.9697 - loss: 0.1013 -
val_accuracy: 0.9931 - val_loss: 0.0451
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 39s 115ms/step - accuracy: 0.9928 - loss: 0.0481 -
val_accuracy: 0.9931 - val_loss: 0.0428
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 39s 113ms/step - accuracy: 0.9928 - loss: 0.0467 -
val_accuracy: 0.9931 - val_loss: 0.0428

Training: ResNet50 | LR: 1e-05 | Dropout: 0.3
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 59s 141ms/step - accuracy: 0.8976 - loss: 0.2478 -
val_accuracy: 0.9931 - val_loss: 0.0487
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 39s 113ms/step - accuracy: 0.9928 - loss: 0.0523 -
val_accuracy: 0.9931 - val_loss: 0.0433
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 40s 115ms/step - accuracy: 0.9928 - loss: 0.0487 -
val_accuracy: 0.9931 - val_loss: 0.0436

Training: ResNet50 | LR: 1e-05 | Dropout: 0.5
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 60s 145ms/step - accuracy: 0.9828 - loss: 0.0944 -
val_accuracy: 0.9931 - val_loss: 0.0423
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 39s 114ms/step - accuracy: 0.9928 - loss: 0.0499 -
val_accuracy: 0.9931 - val_loss: 0.0435
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 39s 114ms/step - accuracy: 0.9928 - loss: 0.0485 -
val_accuracy: 0.9931 - val_loss: 0.0441

Training: VGG16 | LR: 0.0001 | Dropout: 0.3
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 90s 255ms/step - accuracy: 0.9768 - loss: 0.0826 -
val_accuracy: 0.9931 - val_loss: 0.0426
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 247ms/step - accuracy: 0.9928 - loss: 0.0458 -
val_accuracy: 0.9931 - val_loss: 0.0422
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 246ms/step - accuracy: 0.9928 - loss: 0.0455 -
val_accuracy: 0.9931 - val_loss: 0.0419
```

```
Training: VGG16 | LR: 0.0001 | Dropout: 0.5
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 90s 254ms/step - accuracy: 0.9521 - loss: 0.1287 -
val_accuracy: 0.9931 - val_loss: 0.0445
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 247ms/step - accuracy: 0.9928 - loss: 0.0498 -
val_accuracy: 0.9931 - val_loss: 0.0441
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 246ms/step - accuracy: 0.9928 - loss: 0.0483 -
val_accuracy: 0.9931 - val_loss: 0.0435

Training: VGG16 | LR: 1e-05 | Dropout: 0.3
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 90s 254ms/step - accuracy: 0.9502 - loss: 0.2608 -
val_accuracy: 0.9931 - val_loss: 0.1014
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 247ms/step - accuracy: 0.9928 - loss: 0.0761 -
val_accuracy: 0.9931 - val_loss: 0.0550
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 246ms/step - accuracy: 0.9928 - loss: 0.0532 -
val_accuracy: 0.9931 - val_loss: 0.0461

Training: VGG16 | LR: 1e-05 | Dropout: 0.5
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 90s 255ms/step - accuracy: 0.9591 - loss: 0.2391 -
val_accuracy: 0.9931 - val_loss: 0.0968
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 246ms/step - accuracy: 0.9927 - loss: 0.0781 -
val_accuracy: 0.9931 - val_loss: 0.0547
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 85s 246ms/step - accuracy: 0.9928 - loss: 0.0564 -
val_accuracy: 0.9931 - val_loss: 0.0458

Training: MobileNetV2 | LR: 0.0001 | Dropout: 0.3
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 42s 97ms/step - accuracy: 0.9812 - loss: 0.0723 -
val_accuracy: 0.9931 - val_loss: 0.0439
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 20s 58ms/step - accuracy: 0.9928 - loss: 0.0459 -
val_accuracy: 0.9931 - val_loss: 0.0427
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 21s 62ms/step - accuracy: 0.9928 - loss: 0.0426 -
val_accuracy: 0.9931 - val_loss: 0.0421

Training: MobileNetV2 | LR: 0.0001 | Dropout: 0.5
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 42s 95ms/step - accuracy: 0.9874 - loss: 0.0684 -
val_accuracy: 0.9931 - val_loss: 0.0498
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 22s 62ms/step - accuracy: 0.9927 - loss: 0.0487 -
val_accuracy: 0.9931 - val_loss: 0.0485
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 21s 61ms/step - accuracy: 0.9928 - loss: 0.0434 -
```

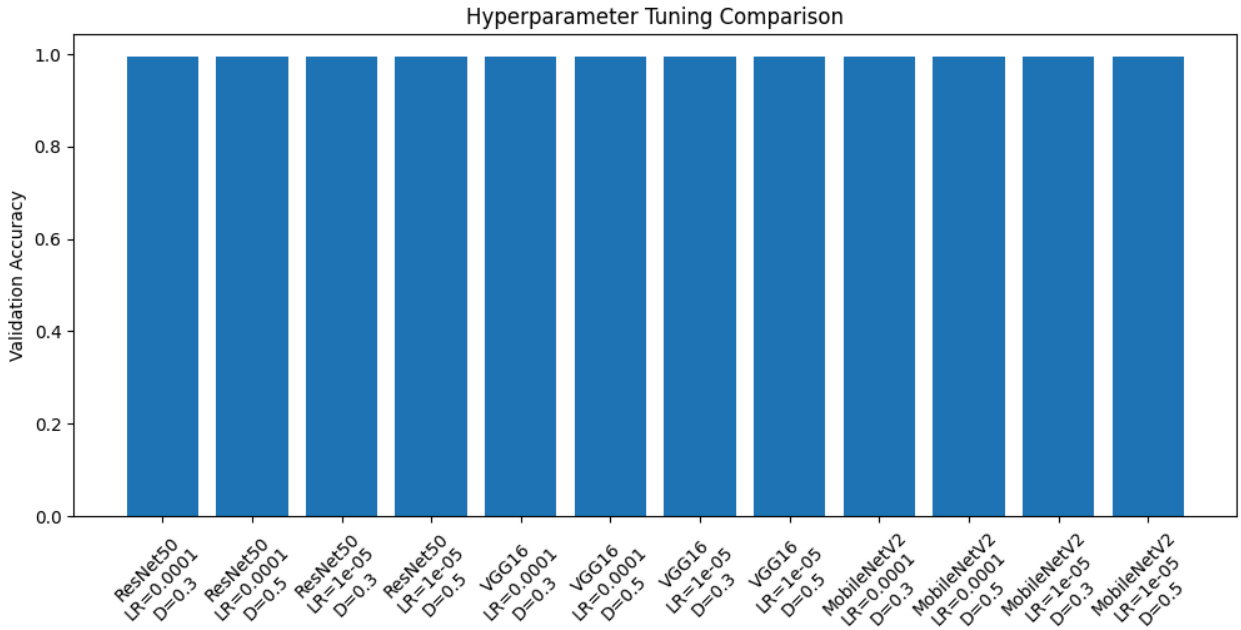
```
val_accuracy: 0.9931 - val_loss: 0.0452

Training: MobileNetV2 | LR: 1e-05 | Dropout: 0.3
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 42s 95ms/step - accuracy: 0.9651 - loss: 0.1286 -
val_accuracy: 0.9931 - val_loss: 0.0452
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 23s 65ms/step - accuracy: 0.9928 - loss: 0.0515 -
val_accuracy: 0.9931 - val_loss: 0.0449
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 21s 61ms/step - accuracy: 0.9928 - loss: 0.0496 -
val_accuracy: 0.9931 - val_loss: 0.0447

Training: MobileNetV2 | LR: 1e-05 | Dropout: 0.5
Epoch 1/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 42s 96ms/step - accuracy: 0.8216 - loss: 0.4158 -
val_accuracy: 0.9931 - val_loss: 0.0578
Epoch 2/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 21s 61ms/step - accuracy: 0.9918 - loss: 0.0663 -
val_accuracy: 0.9931 - val_loss: 0.0490
Epoch 3/3
344/344 ━━━━━━━━━━━━━━━━━━━━ 21s 61ms/step - accuracy: 0.9924 - loss: 0.0603 -
val_accuracy: 0.9931 - val_loss: 0.0486
```

In [30]:

```python
print("\nHyperparameter Tuning Results\n")

for r in results:
    print("Model:", r[0],
          "| LR:", r[1],
          "| Dropout:", r[2],
          "| Accuracy:", r[3])
```

```
Hyperparameter Tuning Results

Model: ResNet50 | LR: 0.0001 | Dropout: 0.3 | Accuracy: 0.9930833578109741
Model: ResNet50 | LR: 0.0001 | Dropout: 0.5 | Accuracy: 0.9930833578109741
Model: ResNet50 | LR: 1e-05 | Dropout: 0.3 | Accuracy: 0.9930833578109741
Model: ResNet50 | LR: 1e-05 | Dropout: 0.5 | Accuracy: 0.9930833578109741
Model: VGG16 | LR: 0.0001 | Dropout: 0.3 | Accuracy: 0.9930833578109741
Model: VGG16 | LR: 0.0001 | Dropout: 0.5 | Accuracy: 0.9930833578109741
Model: VGG16 | LR: 1e-05 | Dropout: 0.3 | Accuracy: 0.9930833578109741
Model: VGG16 | LR: 1e-05 | Dropout: 0.5 | Accuracy: 0.9930833578109741
Model: MobileNetV2 | LR: 0.0001 | Dropout: 0.3 | Accuracy: 0.9930833578109741
Model: MobileNetV2 | LR: 0.0001 | Dropout: 0.5 | Accuracy: 0.9930833578109741
Model: MobileNetV2 | LR: 1e-05 | Dropout: 0.3 | Accuracy: 0.9930833578109741
Model: MobileNetV2 | LR: 1e-05 | Dropout: 0.5 | Accuracy: 0.9930833578109741
```

In [31]:

```python
labels = [f"{r[0]}\nLR={r[1]}\nD={r[2]}" for r in results]
values = [r[3] for r in results]

plt.figure(figsize=(12,5))
plt.bar(labels, values)

plt.title("Hyperparameter Tuning Comparison")
```

```
plt.ylabel("Validation Accuracy")

plt.xticks(rotation=45)
plt.show()
```